



```
In [1]: # =====
# DEADHUNT – FULL ANALYSIS & BETTING OPTIMIZER
# Copy-paste into your notebook and run cell by cell.
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import norm, lognorm, shapiro, kstest, anderson
from scipy.optimize import minimize
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['figure.figsize'] = (14, 6)

df = pd.read_csv('race_data.csv') # <- adjust path if needed
horses = ['Shadowfax', 'Iron Duke', 'Morningstar', 'Red Tide',
          'Gallant Fox', 'Blue Streak', 'Copper Prince', 'Last Chance']

market_probs = {
    'Shadowfax': 0.08, 'Iron Duke': 0.09, 'Morningstar': 0.11,
    'Red Tide': 0.13, 'Gallant Fox': 0.14, 'Blue Streak': 0.15,
    'Copper Prince': 0.14, 'Last Chance': 0.16
}

TAKEOUT = 0.15
BANKROLL = 10_000

# — verify winner column —————
assert (df[horses].idxmin(axis=1) == df['winner']).all(), "winner mismatch!"
print("✓ Winner column verified\n")

# =====
# 1. BASIC STATISTICS
# =====

print("="*60)
print("1. DESCRIPTIVE STATISTICS")
print("="*60)
stats_df = df[horses].describe().T
stats_df['skew'] = df[horses].skew()
stats_df['kurtosis'] = df[horses].kurtosis()
stats_df['cv'] = stats_df['std'] / stats_df['mean'] # coeff of variati
stats_df['iqr'] = df[horses].quantile(0.75) - df[horses].quantile(0.25)
print(stats_df[['mean', 'std', 'cv', 'min', '25%', '50%', '75%', 'max', 'skew', 'kurtos
                ].sort_values('mean').round(4))

# =====
# 2. EMPIRICAL WIN RATES (with 95% CI via binomial)
# =====

print("\n" + "="*60)
print("2. EMPIRICAL WIN RATES vs MARKET")
```

```

print("="*60)
n = len(df)
win_counts = df['winner'].value_counts()
emp = {}
for h in horses:
    p_hat = win_counts.get(h, 0) / n
    se = np.sqrt(p_hat * (1 - p_hat) / n)
    emp[h] = {'emp_prob': p_hat, 'ci_lo': max(0, p_hat - 1.96*se),
              'ci_hi': min(1, p_hat + 1.96*se), 'market': market_probs[h]}

emp_df = pd.DataFrame(emp).T.sort_values('emp_prob', ascending=False)
emp_df['edge_raw'] = emp_df['emp_prob'] - emp_df['market']
emp_df['ev'] = emp_df['emp_prob'] * (1 - TAKEOUT) / emp_df['market'] - 1
print(emp_df.round(4))

fig, ax = plt.subplots()
x = np.arange(len(horses))
ax.bar(x - 0.2, emp_df['emp_prob'], 0.35, label='Empirical', color='steelblue')
ax.bar(x + 0.2, emp_df['market'], 0.35, label='Market', color='coral')
ax.errorbar(x - 0.2, emp_df['emp_prob'],
            yerr=[emp_df['emp_prob']-emp_df['ci_lo'], emp_df['ci_hi']-emp_df['emp_prob']],
            fmt='none', color='black', capsize=4)
ax.set_xticks(x); ax.set_xticklabels(emp_df.index, rotation=30, ha='right')
ax.set_title("Empirical Win Rate vs Market (with 95% CI)"); ax.legend()
plt.tight_layout(); plt.show()

# =====
# 3. DISTRIBUTION FITTING – Normal, Log-Normal, Gamma
# =====

print("\n" + "="*60)
print("3. DISTRIBUTION FITTING")
print("="*60)

from scipy.stats import gamma as gamma_dist

fit_results = {}
for h in horses:
    t = df[h].values
    # fit all three
    mu, sigma = t.mean(), t.std()
    norm_ll = norm.logpdf(t, mu, sigma).sum()

    sh_ln, loc_ln, sc_ln = lognorm.fit(t, floc=0)
    lognorm_ll = lognorm.logpdf(t, sh_ln, loc_ln, sc_ln).sum()

    sh_g, loc_g, sc_g = gamma_dist.fit(t, floc=0)
    gamma_ll = gamma_dist.logpdf(t, sh_g, loc_g, sc_g).sum()

    best = max([('Normal', norm_ll), ('LogNormal', lognorm_ll), ('Gamma', gamma_ll)],
               key=lambda x: x[1])

    # Shapiro-Wilk
    _, sw_p = shapiro(t)

```

```

# KS vs Normal
_, ks_p = kstest(t, 'norm', args=(mu, sigma))

fit_results[h] = {
    'best_fit': best[0],
    'norm_ll': round(norm_ll,2), 'lognorm_ll': round(lognorm_ll,2), 'gamma
    'shapiro_p': round(sw_p,4), 'ks_p': round(ks_p,4),
    'norm_params': (mu, sigma),
    'ln_params': (sh_ln, loc_ln, sc_ln),
    'gam_params': (sh_g, loc_g, sc_g),
}
print(f"{h:<15} → {best[0]:<10} (Normal LL={norm_ll:.1f}, LN LL={lognorm_
    f"Gamma LL={gamma_ll:.1f}) Shapiro p={sw_p:.4f}")

# =====
# 4. QQ-PLOTS FOR EACH HORSE
# =====
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, h in zip(axes.flatten(), horses):
    stats.probplot(df[h], dist='norm', plot=ax)
    ax.set_title(f"{h} – QQ (Normal)")
plt.suptitle("QQ-Plots (Normal)", y=1.01); plt.tight_layout(); plt.show()

# =====
# 5. KDE + FITTED DISTRIBUTION OVERLAY
# =====
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, h in zip(axes.flatten(), horses):
    t = df[h].values
    ax.hist(t, bins=30, density=True, alpha=0.4, color='steelblue')
    xs = np.linspace(t.min(), t.max(), 300)
    # Normal
    mu, sigma = fit_results[h]['norm_params']
    ax.plot(xs, norm.pdf(xs, mu, sigma), 'r-', lw=1.5, label='Normal')
    # LogNormal
    sh, loc, sc = fit_results[h]['ln_params']
    ax.plot(xs, lognorm.pdf(xs, sh, loc, sc), 'g--', lw=1.5, label='LogNormal')
    ax.set_title(h); ax.legend(fontsize=7)
plt.suptitle("Histogram + Fitted Distributions", y=1.01); plt.tight_layout();

# =====
# 6. CORRELATION ANALYSIS
# =====
print("\n" + "="*60)
print("6. CORRELATION BETWEEN HORSE TIMES")
print("="*60)
corr = df[horses].corr()
print(corr.round(3))
# Note: near-zero correlations → horses are independent → simulate independent

fig, ax = plt.subplots(figsize=(10,8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', ax=ax,
            vmin=-1, vmax=1, center=0)

```

```

ax.set_title("Correlation Between Finishing Times"); plt.tight_layout(); plt.s

# =====
# 7. LARGE MONTE CARLO SIMULATION (best-fit per horse)
# =====
print("\n" + "="*60)
print("7. MONTE CARLO SIMULATION (500,000 races)")
print("="*60)
N_SIMS = 500_000
np.random.seed(42)
sim = np.zeros((N_SIMS, len(horses)))
for i, h in enumerate(horses):
    fr = fit_results[h]
    if fr['best_fit'] == 'Normal':
        mu, sig = fr['norm_params']
        sim[:, i] = np.random.normal(mu, sig, N_SIMS)
    elif fr['best_fit'] == 'LogNormal':
        sh, loc, sc = fr['ln_params']
        sim[:, i] = lognorm.rvs(sh, loc, sc, size=N_SIMS)
    else:
        sh, loc, sc = fr['gam_params']
        sim[:, i] = gamma_dist.rvs(sh, loc, sc, size=N_SIMS)

win_idx = np.argmin(sim, axis=1)
mc_probs = {horses[i]: (win_idx == i).mean() for i in range(len(horses))}
print(f"{'Horse':<15} {'MC Prob':>9} {'Market':>9} {'EV':>9}")
for h in horses:
    p = mc_probs[h]
    m = market_probs[h]
    ev = p * (1 - TAKEOUT) / m - 1
    print(f"{h:<15} {p:>9.4f} {m:>9.4f} {ev:>+9.4f}")

# =====
# 8. BOOTSTRAP CONFIDENCE INTERVALS ON MC PROBS
# =====
print("\n" + "="*60)
print("8. BOOTSTRAP CI ON MC WIN PROBABILITIES")
print("="*60)
N_BOOT = 500
boot_probs = {h: [] for h in horses}
sample_size = 500
for _ in range(N_BOOT):
    sample = df[horses].sample(n=sample_size, replace=True).values
    w_idx = np.argmin(sample, axis=1)
    for i, h in enumerate(horses):
        boot_probs[h].append((w_idx == i).mean())

print(f"{'Horse':<15} {'Mean':>8} {'CI_Lo':>8} {'CI_Hi':>8}")
boot_ci = {}
for h in horses:
    arr = np.array(boot_probs[h])
    lo, hi = np.percentile(arr, [2.5, 97.5])
    boot_ci[h] = (arr.mean(), lo, hi)

```

```

    print(f"{h:<15} {arr.mean():>8.4f} {lo:>8.4f} {hi:>8.4f}")

# =====
# 9. HEAD-TO-HEAD WIN MATRIX
# =====
print("\n" + "="*60)
print("9. HEAD-TO-HEAD DOMINANCE MATRIX (% horse A beats horse B)")
print("="*60)
h2h = pd.DataFrame(index=horses, columns=horses, dtype=float)
for i, ha in enumerate(horses):
    for j, hb in enumerate(horses):
        h2h.loc[ha, hb] = (df[ha] < df[hb]).mean()
print(h2h.round(3))

fig, ax = plt.subplots(figsize=(10,8))
sns.heatmap(h2h.astype(float), annot=True, fmt='.2f', cmap='RdYlGn', ax=ax, vm
ax.set_title("Head-to-Head: P(row beats col)"); plt.tight_layout(); plt.show()

# =====
# 10. PLACE / SHOW PROBABILITIES (top-2, top-3 finishes)
# =====
print("\n" + "="*60)
print("10. TOP-2 AND TOP-3 FINISH PROBABILITIES (from simulation)")
print("="*60)
rank_matrix = np.argsort(sim, axis=1) # shape (N_SIMS, 8), rank[i,j]
# rank_matrix[:,i] = ranks of horse i across all sims
ranks = np.zeros_like(sim, dtype=int)
for i in range(N_SIMS):
    ranks[i, rank_matrix[i]] = np.arange(len(horses))

print(f"{'Horse':<15} {'Win':>7} {'Top-2':>7} {'Top-3':>7}")
for i, h in enumerate(horses):
    w = (ranks[:, i] == 0).mean()
    t2 = (ranks[:, i] <= 1).mean()
    t3 = (ranks[:, i] <= 2).mean()
    print(f"{h:<15} {w:>7.3f} {t2:>7.3f} {t3:>7.3f}")

# =====
# 11. STREAK ANALYSIS – recent form (last 50 / last 100 races)
# =====
print("\n" + "="*60)
print("11. RECENT FORM (last 50 vs last 100 races)")
print("="*60)
for window, label in [(50, 'Last 50'), (100, 'Last 100'), (500, 'All 500')]:
    subset = df.tail(window)
    wc = subset['winner'].value_counts()
    print(f"\n {label}:")
    for h in horses:
        print(f"      {h:<15}: {wc.get(h,0):>3} wins ({wc.get(h,0)/window:.1%})

# Rolling win rate chart
fig, ax = plt.subplots(figsize=(14,5))
for h in horses:

```

```

    roll = (df['winner'] == h).rolling(50).mean()
    ax.plot(roll, label=h)
ax.axhline(1/8, color='black', ls='--', lw=1, label='Uniform 12.5%')
ax.set_title("50-Race Rolling Win Rate"); ax.legend(ncol=4, fontsize=8)
plt.tight_layout(); plt.show()

# =====
# 12. PACE / FINISHING TIME TRENDS OVER RACES
# =====
print("\n" + "="*60)
print("12. PACE TREND – is any horse getting faster/slower?")
print("="*60)
for h in horses:
    slope, intercept, r, p, se = stats.linregress(df['race_id'], df[h])
    print(f"{h:<15} slope={slope:+.5f} s/race    p={p:.4f} {'*SIGNIFICANT*' if

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, h in zip(axes.flatten(), horses):
    ax.scatter(df['race_id'], df[h], s=2, alpha=0.4)
    m, b, *_ = stats.linregress(df['race_id'], df[h])
    ax.plot(df['race_id'], m*df['race_id']+b, 'r-', lw=1.5)
    ax.set_title(h); ax.set_xlabel('Race'); ax.set_ylabel('Time (s)')
plt.suptitle("Finishing Time Trend per Horse", y=1.01); plt.tight_layout(); pl

# =====
# 13. VOLATILITY ANALYSIS – variance of rank positions
# =====
print("\n" + "="*60)
print("13. RANK VOLATILITY (std dev of finishing rank across 500 races)")
print("="*60)
rank_df = df[horses].rank(axis=1) # 1 = fastest
print(f"{'Horse':<15} {'Mean Rank':>10} {'Std Rank':>10} (lower mean & std =
for h in horses:
    print(f"{h:<15} {rank_df[h].mean():>10.3f} {rank_df[h].std():>10.3f}")

# =====
# 14. KELLY CRITERION – OPTIMAL BET SIZING
# =====
print("\n" + "="*60)
print("14. KELLY CRITERION ANALYSIS")
print("="*60)

# Use MC probs
evs = {h: mc_probs[h] * (1-TAKEOUT) / market_probs[h] - 1 for h in horses}
pos_ev = [h for h in horses if evs[h] > 0]

print(f"\nPositive EV horses: {pos_ev}")
print(f"\n{'Horse':<15} {'p_true':>8} {'f_mkt':>8} {'EV':>8}")
for h in horses:
    print(f"{h:<15} {mc_probs[h]:>8.4f} {market_probs[h]:>8.4f} {evs[h]:>+8.4f

# Simultaneous Kelly for parimutuel (Smoczyński & Tomkins formula)
sum_p      = sum(mc_probs[h] for h in pos_ev)

```

```

sum_f_adj = sum(market_probs[h] / (1-TAKEOUT) for h in pos_ev)
Q         = (1 - sum_p) / (1 - sum_f_adj)

kelly_fracs = {}
for h in pos_ev:
    f = mc_probs[h] - (market_probs[h] / (1-TAKEOUT)) * Q
    kelly_fracs[h] = max(0, f)

print(f"\nSimultaneous Kelly fractions:")
for h in horses:
    kf = kelly_fracs.get(h, 0)
    print(f" {h:<15}: {kf:.4f} → £{kf*BANKROLL:,.0f}")

# =====
# 15. STRATEGY COMPARISON & SIMULATION
# =====

print("\n" + "="*60)
print("15. STRATEGY COMPARISON (simulated P&L)")
print("="*60)

def sim_strategy(stakes_dict, n=100_000):
    """Simulate average profit per race."""
    stakes = np.array([stakes_dict.get(h, 0) for h in horses])
    payouts = stakes * (1 - TAKEOUT) / np.array([market_probs[h] for h in horses])
    profits_per_outcome = payouts - stakes.sum() # net if that horse wins
    # add back the stake on the winning horse
    profits_per_outcome = payouts - stakes.sum()

    # simulate
    np.random.seed(0)
    sim2 = np.zeros((n, len(horses)))
    for i, h in enumerate(horses):
        fr = fit_results[h]
        if fr['best_fit'] == 'Normal':
            mu, sig = fr['norm_params']
            sim2[:, i] = np.random.normal(mu, sig, n)
        elif fr['best_fit'] == 'LogNormal':
            sh, loc, sc = fr['ln_params']
            sim2[:, i] = lognorm.rvs(sh, loc, sc, size=n)
        else:
            sh, loc, sc = fr['gam_params']
            sim2[:, i] = gamma_dist.rvs(sh, loc, sc, size=n)

    winners2 = np.argmax(sim2, axis=1)
    total_stake = stakes.sum()
    profits = np.array([payouts[w] - total_stake for w in winners2])
    return profits.mean(), profits.std(), (profits > 0).mean()

strategies = {}

# S1: All-in on best EV horse
best_ev_horse = max(evs, key=evs.get)
strategies['AllIn_BestEV'] = {best_ev_horse: BANKROLL}

```

```

# S2: Pure Kelly
total_kf = sum(kelly_fracs.values())
strategies['Full_Kelly'] = {h: kelly_fracs.get(h,0) * BANKROLL for h in horses}

# S3: Half Kelly (safer)
strategies['Half_Kelly'] = {h: kelly_fracs.get(h,0) * BANKROLL * 0.5 for h in horses}

# S4: Proportional to true win prob (pos EV only)
strategies['Prob_Weight'] = {h: (mc_probs[h]/sum_p)*BANKROLL if h in pos_ev else 0 for h in horses}

# S5: 70% Norm Kelly + 30% Prob
norm_kelly = {h: (kelly_fracs.get(h,0)/total_kf)*BANKROLL if total_kf>0 else 0 for h in horses}
strategies['Clubbed_70_30'] = {h: 0.7*norm_kelly[h] + 0.3*strategies['Prob_Weight'][h] for h in horses}

# S6: Equal split on pos EV
strategies['Equal_PosEV'] = {h: BANKROLL/len(pos_ev) if h in pos_ev else 0 for h in horses}

print(f"\n{'Strategy':<20} {'Avg Profit':>12} {'Std Dev':>10} {'Win Rate':>10}")
best_strat, best_avg = None, -np.inf
for name, stk in strategies.items():
    avg, std, wr = sim_strategy(stk)
    print(f"{name:<20} {avg:>+12.2f} {std:>10.2f} {wr:>10.2%}")
    if avg > best_avg:
        best_avg, best_strat = avg, name

print(f"\n★ Best strategy by avg profit: {best_strat} (£{best_avg:+.2f}/race)")

# =====
# 16. FINAL RECOMMENDED STAKES
# =====

print("\n" + "="*60)
print("16. FINAL RECOMMENDED STAKES")
print("="*60)

final = strategies[best_strat]
# round and ensure sum <= BANKROLL
final_int = {h: int(v) for h, v in final.items()}
leftover = BANKROLL - sum(final_int.values())
final_int[best_ev_horse] = final_int.get(best_ev_horse, 0) + leftover

print(f"\nBased on strategy: {best_strat}")
print(f"{'Horse':<15} {'Stake £':>10} {'Payout if Win £':>18} {'True P':>8} {'')
for h in horses:
    stk = final_int.get(h, 0)
    payout = stk * (1-TAKEOUT) / market_probs[h] if stk > 0 else 0
    print(f"{h:<15} {stk:>10} {payout:>18,.0f} {mc_probs[h]:>8.3f} {market_probs[h]:>8.3f}")

print(f"\nTotal staked: £{sum(final_int.values()):,}")
print(f"Expected profit per race: £{best_avg:+.2f}")
print(f"Expected profit over 10k races: £{best_avg*10000:+,.0f}")

# =====

```

```

# 17. SENSITIVITY ANALYSIS – how robust is the edge?
# =====
print("\n" + "="*60)
print("17. SENSITIVITY: EV under different true-prob assumptions")
print("="*60)
# Vary the win prob ± 20% and see if edge flips
for h in pos_ev:
    for factor in [0.8, 0.9, 1.0, 1.1, 1.2]:
        adj_p = mc_probs[h] * factor
        adj_ev = adj_p * (1-TAKEOUT) / market_probs[h] - 1
        flag = "✓" if adj_ev > 0 else "✗"
        print(f" {h:<15} p×{factor:.1f} → p={adj_p:.3f} EV={adj_ev:+.4f} {f
print()

```

✓ Winner column verified

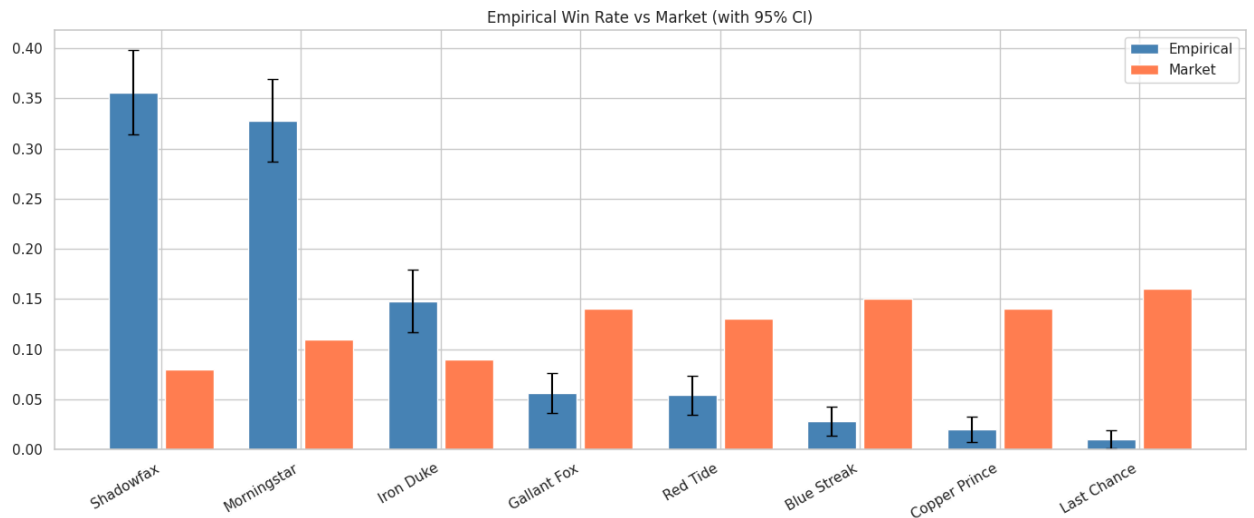
1. DESCRIPTIVE STATISTICS

	mean	std	cv	min	25%	50%	75%	\
Shadowfax	66.6650	2.5906	0.0389	59.2294	64.9529	66.5261	68.1770	
Morningstar	67.3246	4.1333	0.0614	57.1131	64.3836	67.4458	70.1330	
Iron Duke	68.0644	2.3278	0.0342	61.4301	66.5203	68.1262	69.6420	
Red Tide	70.1464	3.1677	0.0452	61.0767	67.9535	70.1582	72.1328	
Gallant Fox	70.8782	3.5019	0.0494	61.6270	68.5770	70.7860	73.1541	
Blue Streak	72.3347	4.0526	0.0560	59.1063	69.6590	72.2686	74.8370	
Copper Prince	74.3899	4.8446	0.0651	62.2044	70.9100	74.4080	77.5323	
Last Chance	77.5895	6.3846	0.0823	62.2300	72.7668	77.2625	81.5834	

	max	skew	kurtosis	iqr
Shadowfax	75.4553	0.3954	0.5607	3.2241
Morningstar	79.2530	0.1347	-0.2610	5.7494
Iron Duke	75.0635	-0.0080	0.0649	3.1217
Red Tide	79.9278	-0.0602	-0.0192	4.1793
Gallant Fox	81.6355	0.1428	-0.1051	4.5770
Blue Streak	85.3488	0.1042	0.3486	5.1780
Copper Prince	91.5262	0.2304	0.0084	6.6223
Last Chance	98.9646	0.3767	-0.0620	8.8166

2. EMPIRICAL WIN RATES vs MARKET

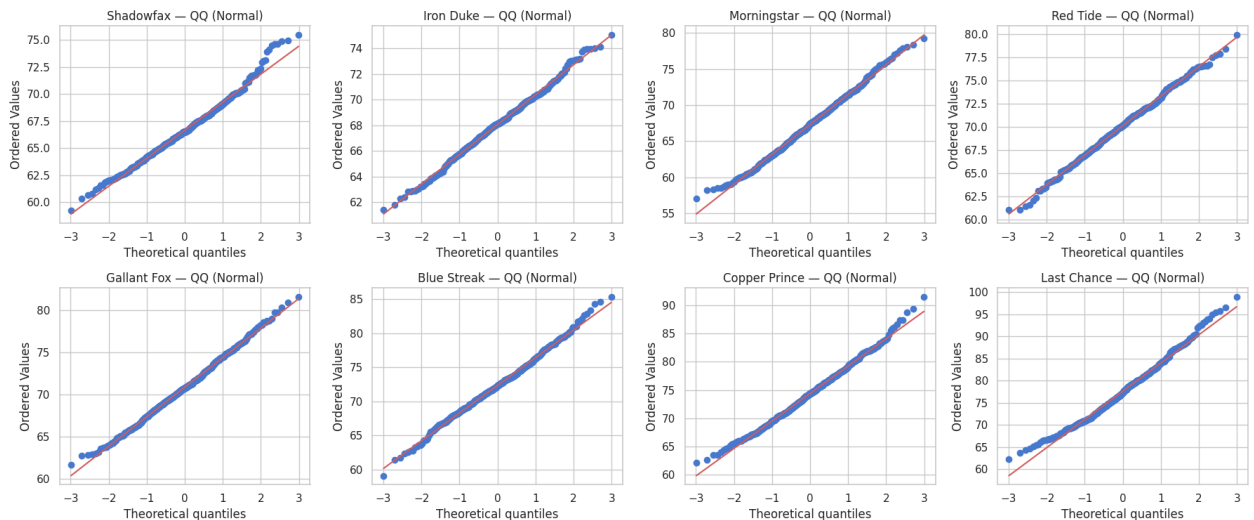
	emp_prob	ci_lo	ci_hi	market	edge_raw	ev
Shadowfax	0.356	0.3140	0.3980	0.08	0.276	2.7825
Morningstar	0.328	0.2868	0.3692	0.11	0.218	1.5345
Iron Duke	0.148	0.1169	0.1791	0.09	0.058	0.3978
Gallant Fox	0.056	0.0358	0.0762	0.14	-0.084	-0.6600
Red Tide	0.054	0.0342	0.0738	0.13	-0.076	-0.6469
Blue Streak	0.028	0.0135	0.0425	0.15	-0.122	-0.8413
Copper Prince	0.020	0.0077	0.0323	0.14	-0.120	-0.8786
Last Chance	0.010	0.0013	0.0187	0.16	-0.150	-0.9469



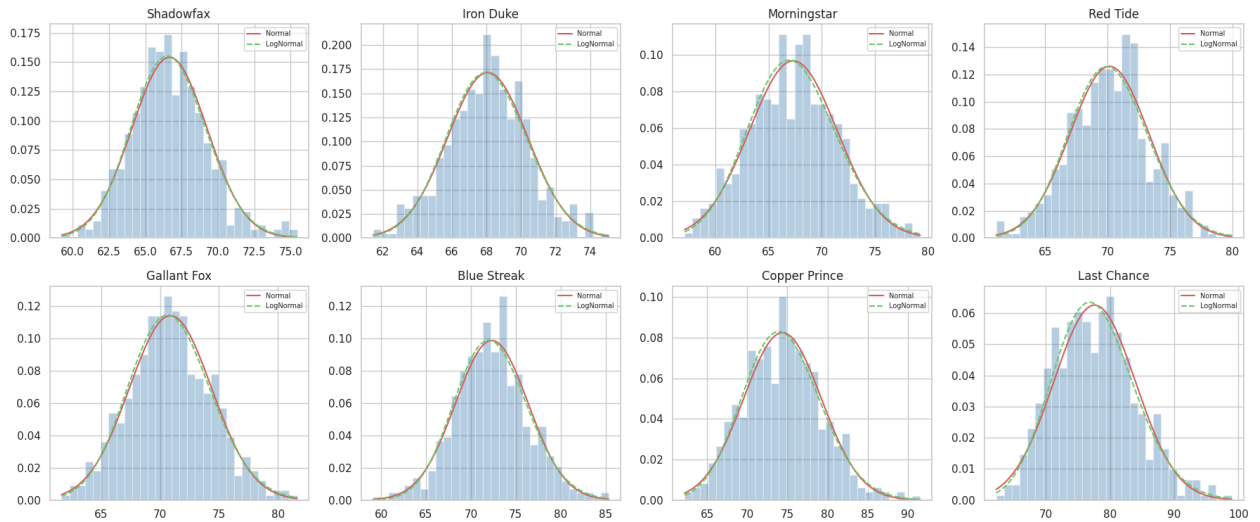
3. DISTRIBUTION FITTING

Shadowfax	→ LogNormal	(Normal LL=-1184.9, LN LL=-1181.8, Gamma LL=-1182.7)
Shapiro	p=0.0006	
Iron Duke	→ Normal	(Normal LL=-1131.4, LN LL=-1131.9, Gamma LL=-1131.7)
Shapiro	p=0.4648	
Morningstar	→ LogNormal	(Normal LL=-1418.5, LN LL=-1417.6, Gamma LL=-1417.6)
Shapiro	p=0.1785	
Red Tide	→ Normal	(Normal LL=-1285.5, LN LL=-1286.9, Gamma LL=-1286.3)
Shapiro	p=0.7733	
Gallant Fox	→ LogNormal	(Normal LL=-1335.6, LN LL=-1334.7, Gamma LL=-1334.8)
Shapiro	p=0.5087	
Blue Streak	→ Gamma	(Normal LL=-1408.6, LN LL=-1408.6, Gamma LL=-1408.3)
Shapiro	p=0.4217	
Copper Prince	→ LogNormal	(Normal LL=-1497.9, LN LL=-1495.6, Gamma LL=-1495.6)
Shapiro	p=0.1235	
Last Chance	→ LogNormal	(Normal LL=-1635.9, LN LL=-1630.3, Gamma LL=-1630.3)
Shapiro	p=0.0006	

QQ-Plots (Normal)



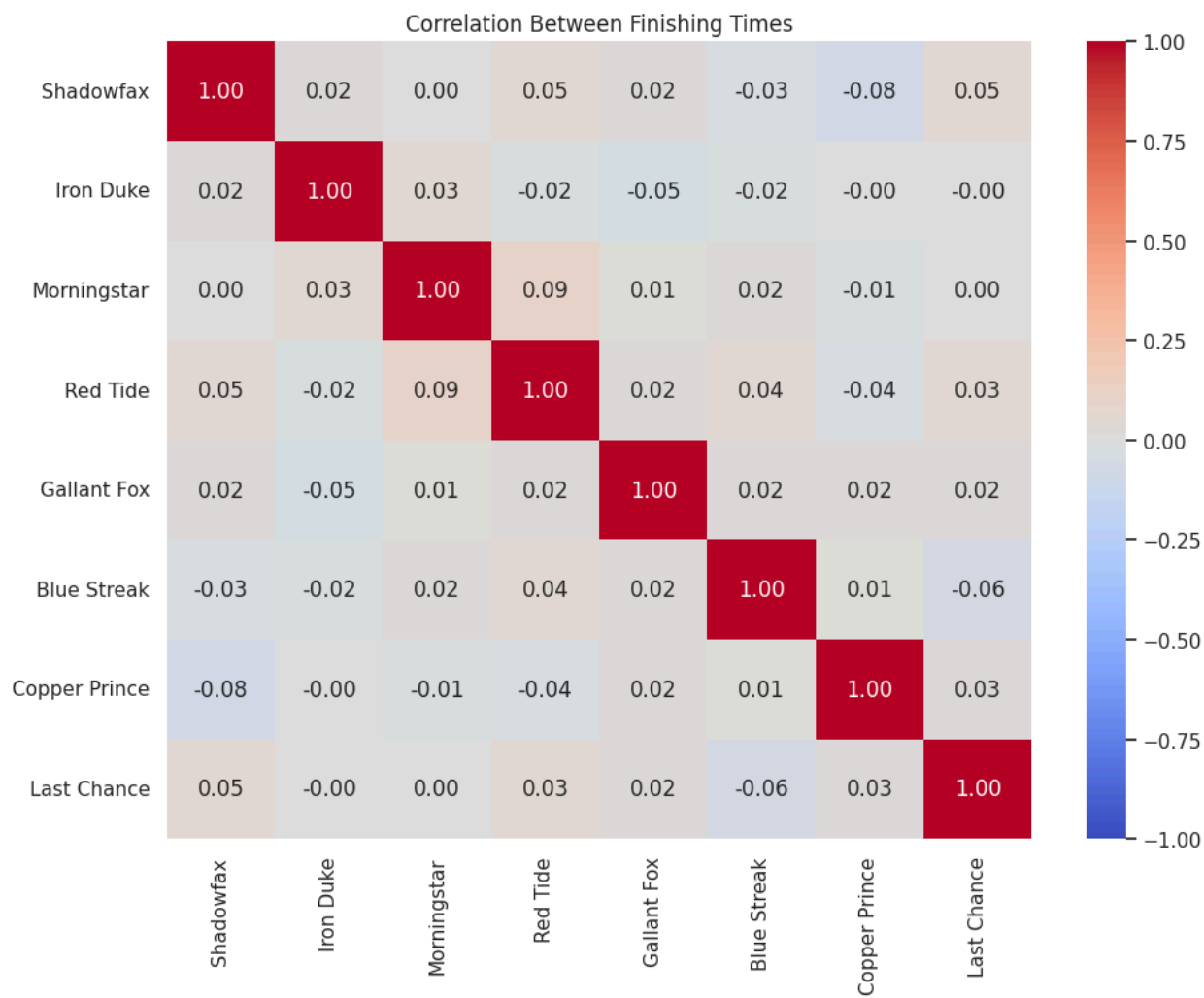
Histogram + Fitted Distributions



6. CORRELATION BETWEEN HORSE TIMES

	Shadowfax	Iron Duke	Morningstar	Red Tide	Gallant Fox \
Shadowfax	1.000	0.021	0.003	0.046	0.023
Iron Duke	0.021	1.000	0.032	-0.022	-0.050
Morningstar	0.003	0.032	1.000	0.090	0.009
Red Tide	0.046	-0.022	0.090	1.000	0.019
Gallant Fox	0.023	-0.050	0.009	0.019	1.000
Blue Streak	-0.030	-0.017	0.020	0.044	0.016
Copper Prince	-0.078	-0.003	-0.011	-0.035	0.017
Last Chance	0.053	-0.004	0.001	0.034	0.017

	Blue Streak	Copper Prince	Last Chance
Shadowfax	-0.030	-0.078	0.053
Iron Duke	-0.017	-0.003	-0.004
Morningstar	0.020	-0.011	0.001
Red Tide	0.044	-0.035	0.034
Gallant Fox	0.016	0.017	0.017
Blue Streak	1.000	0.009	-0.056
Copper Prince	0.009	1.000	0.027
Last Chance	-0.056	0.027	1.000



=====

7. MONTE CARLO SIMULATION (500,000 races)

=====

Horse	MC Prob	Market	EV
Shadowfax	0.3304	0.0800	+2.5103
Iron Duke	0.1396	0.0900	+0.3180
Morningstar	0.3314	0.1100	+1.5612
Red Tide	0.0664	0.1300	-0.5660
Gallant Fox	0.0535	0.1400	-0.6750
Blue Streak	0.0377	0.1500	-0.7865
Copper Prince	0.0233	0.1400	-0.8588
Last Chance	0.0178	0.1600	-0.9055

=====

8. BOOTSTRAP CI ON MC WIN PROBABILITIES

=====

Horse	Mean	CI_Lo	CI_Hi
Shadowfax	0.3562	0.3090	0.4000
Iron Duke	0.1480	0.1180	0.1780
Morningstar	0.3281	0.2889	0.3680
Red Tide	0.0543	0.0350	0.0750
Gallant Fox	0.0559	0.0370	0.0770
Blue Streak	0.0277	0.0140	0.0420
Copper Prince	0.0197	0.0090	0.0340
Last Chance	0.0100	0.0020	0.0180

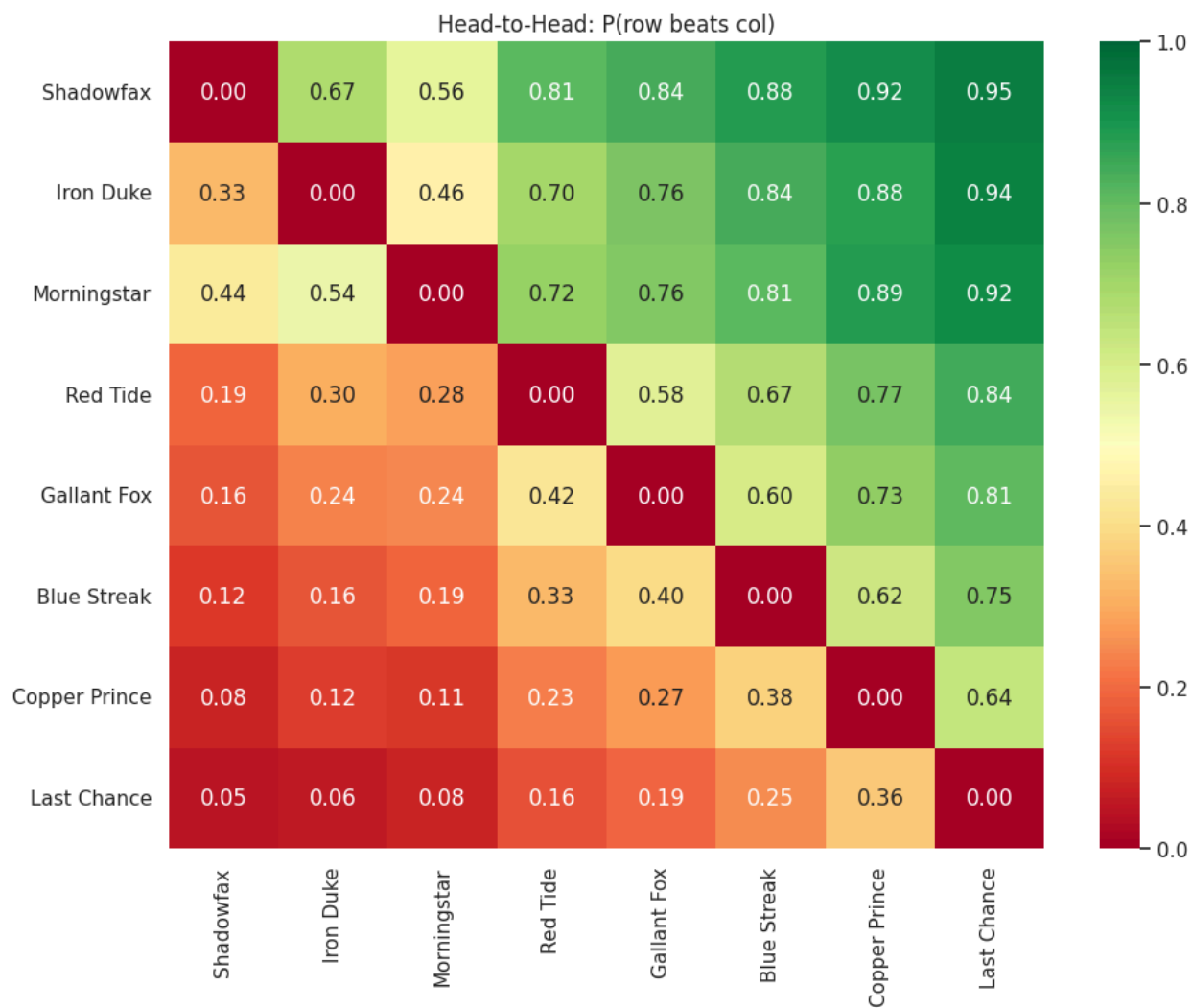
=====

9. HEAD-TO-HEAD DOMINANCE MATRIX (% horse A beats horse B)

=====

	Shadowfax	Iron Duke	Morningstar	Red Tide	Gallant Fox \
Shadowfax	0.000	0.672	0.560	0.810	0.838
Iron Duke	0.328	0.000	0.458	0.698	0.762
Morningstar	0.440	0.542	0.000	0.720	0.756
Red Tide	0.190	0.302	0.280	0.000	0.576
Gallant Fox	0.162	0.238	0.244	0.424	0.000
Blue Streak	0.118	0.164	0.190	0.332	0.398
Copper Prince	0.080	0.122	0.114	0.234	0.272
Last Chance	0.046	0.056	0.078	0.160	0.192

	Blue Streak	Copper Prince	Last Chance
Shadowfax	0.882	0.920	0.954
Iron Duke	0.836	0.878	0.944
Morningstar	0.810	0.886	0.922
Red Tide	0.668	0.766	0.840
Gallant Fox	0.602	0.728	0.808
Blue Streak	0.000	0.622	0.750
Copper Prince	0.378	0.000	0.644
Last Chance	0.250	0.356	0.000



=====

10. TOP-2 AND TOP-3 FINISH PROBABILITIES (from simulation)

=====

Horse	Win	Top-2	Top-3
Shadowfax	0.330	0.613	0.796
Iron Duke	0.140	0.369	0.620
Morningstar	0.331	0.513	0.655
Red Tide	0.066	0.172	0.318
Gallant Fox	0.054	0.140	0.261
Blue Streak	0.038	0.095	0.175
Copper Prince	0.023	0.058	0.108
Last Chance	0.018	0.039	0.067

=====

11. RECENT FORM (last 50 vs last 100 races)

=====

Last 50:

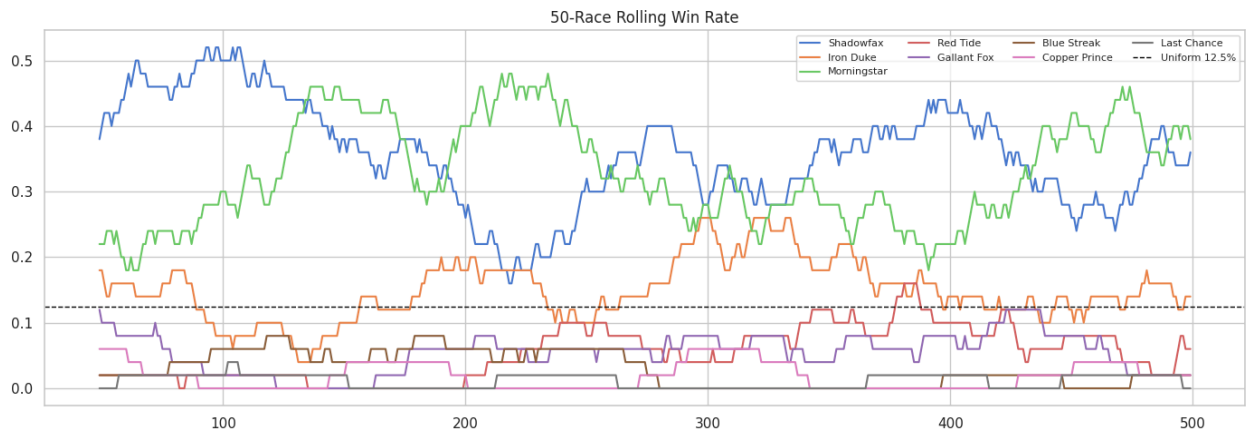
Shadowfax	:	18 wins	(36.0%)
Iron Duke	:	7 wins	(14.0%)
Morningstar	:	19 wins	(38.0%)
Red Tide	:	3 wins	(6.0%)
Gallant Fox	:	1 wins	(2.0%)
Blue Streak	:	1 wins	(2.0%)
Copper Prince	:	1 wins	(2.0%)
Last Chance	:	0 wins	(0.0%)

Last 100:

Shadowfax	:	32 wins	(32.0%)
Iron Duke	:	13 wins	(13.0%)
Morningstar	:	39 wins	(39.0%)
Red Tide	:	7 wins	(7.0%)
Gallant Fox	:	5 wins	(5.0%)
Blue Streak	:	1 wins	(1.0%)
Copper Prince	:	2 wins	(2.0%)
Last Chance	:	1 wins	(1.0%)

All 500:

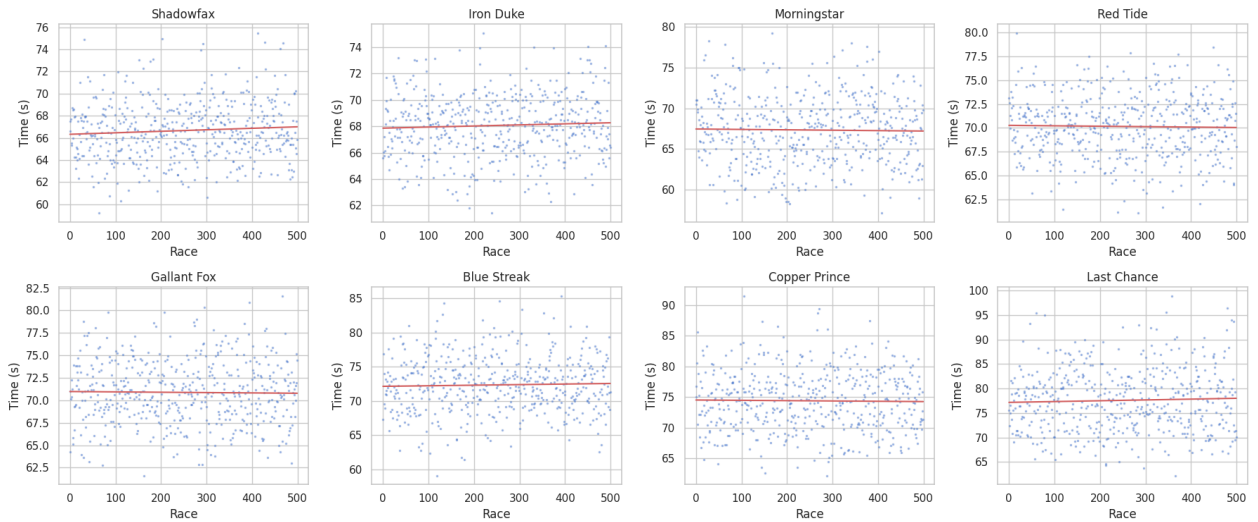
Shadowfax	:	178 wins	(35.6%)
Iron Duke	:	74 wins	(14.8%)
Morningstar	:	164 wins	(32.8%)
Red Tide	:	27 wins	(5.4%)
Gallant Fox	:	28 wins	(5.6%)
Blue Streak	:	14 wins	(2.8%)
Copper Prince	:	10 wins	(2.0%)
Last Chance	:	5 wins	(1.0%)



12. PACE TREND – is any horse getting faster/slower?

Shadowfax	slope=+0.00135 s/race	p=0.0926
Iron Duke	slope=+0.00082 s/race	p=0.2584
Morningstar	slope=-0.00051 s/race	p=0.6937
Red Tide	slope=-0.00045 s/race	p=0.6505
Gallant Fox	slope=-0.00038 s/race	p=0.7279
Blue Streak	slope=+0.00083 s/race	p=0.5077
Copper Prince	slope=-0.00051 s/race	p=0.7365
Last Chance	slope=+0.00167 s/race	p=0.3994

Finishing Time Trend per Horse



=====

13. RANK VOLATILITY (std dev of finishing rank across 500 races)

=====

Horse	Mean Rank	Std Rank	(lower mean & std = consistently fast)
Shadowfax	2.364	1.438	
Iron Duke	3.096	1.500	
Morningstar	2.924	1.892	
Red Tide	4.378	1.772	
Gallant Fox	4.794	1.830	
Blue Streak	5.426	1.848	
Copper Prince	6.156	1.798	
Last Chance	6.862	1.646	

=====

14. KELLY CRITERION ANALYSIS

=====

Positive EV horses: ['Shadowfax', 'Iron Duke', 'Morningstar']

Horse	p_true	f_mkt	EV
Shadowfax	0.3304	0.0800	+2.5103
Iron Duke	0.1396	0.0900	+0.3180
Morningstar	0.3314	0.1100	+1.5612
Red Tide	0.0664	0.1300	-0.5660
Gallant Fox	0.0535	0.1400	-0.6750
Blue Streak	0.0377	0.1500	-0.7865
Copper Prince	0.0233	0.1400	-0.8588
Last Chance	0.0178	0.1600	-0.9055

Simultaneous Kelly fractions:

Shadowfax	:	0.3025	→	£3,025
Iron Duke	:	0.1082	→	£1,082
Morningstar	:	0.2931	→	£2,931
Red Tide	:	0.0000	→	£0
Gallant Fox	:	0.0000	→	£0
Blue Streak	:	0.0000	→	£0
Copper Prince	:	0.0000	→	£0
Last Chance	:	0.0000	→	£0

=====

15. STRATEGY COMPARISON (simulated P&L)

=====

Strategy	Avg Profit	Std Dev	Win Rate
AllIn_BestEV	+25184.69	50004.11	33.12%
Full_Kelly	+12534.54	11981.43	80.18%
Half_Kelly	+6267.27	5990.71	80.18%
Prob_Weight	+17387.11	16170.24	80.18%
Clubbed_70_30	+17682.67	16762.62	80.18%
Equal_PosEV	+14649.14	12872.74	80.18%

★ Best strategy by avg profit: AllIn_BestEV (£+25184.69/race)

=====

16. FINAL RECOMMENDED STAKES

Based on strategy: AllIn_BestEV

Horse	Stake £	Payout if Win £	True P	Market P	EV
Shadowfax	10,000	106,250	0.330	0.080	+2.510
Iron Duke	0	0	0.140	0.090	+0.318
Morningstar	0	0	0.331	0.110	+1.561
Red Tide	0	0	0.066	0.130	-0.566
Gallant Fox	0	0	0.054	0.140	-0.675
Blue Streak	0	0	0.038	0.150	-0.787
Copper Prince	0	0	0.023	0.140	-0.859
Last Chance	0	0	0.018	0.160	-0.906

Total staked: £10,000

Expected profit per race: £+25184.69

Expected profit over 10k races: £+251,846,875

17. SENSITIVITY: EV under different true-prob assumptions

Shadowfax	p×0.8 → p=0.264	EV=+1.8082	✓
Shadowfax	p×0.9 → p=0.297	EV=+2.1593	✓
Shadowfax	p×1.0 → p=0.330	EV=+2.5103	✓
Shadowfax	p×1.1 → p=0.363	EV=+2.8613	✓
Shadowfax	p×1.2 → p=0.396	EV=+3.2124	✓
Iron Duke	p×0.8 → p=0.112	EV=+0.0544	✓
Iron Duke	p×0.9 → p=0.126	EV=+0.1862	✓
Iron Duke	p×1.0 → p=0.140	EV=+0.3180	✓
Iron Duke	p×1.1 → p=0.154	EV=+0.4499	✓
Iron Duke	p×1.2 → p=0.167	EV=+0.5817	✓
Morningstar	p×0.8 → p=0.265	EV=+1.0489	✓
Morningstar	p×0.9 → p=0.298	EV=+1.3050	✓
Morningstar	p×1.0 → p=0.331	EV=+1.5612	✓
Morningstar	p×1.1 → p=0.365	EV=+1.8173	✓
Morningstar	p×1.2 → p=0.398	EV=+2.0734	✓

In []: